

HelixCode — Open Issues Tracker

Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).

Authoritative resumption ledger: docs/CONTINUATION.md (CONST-044). This file complements it with item-level granularity for currently-open work.

Status vocabulary (closed set): Queued | In progress | Ready for testing | In testing | Reopened | Fixed/Implemented/Completed (→ Fixed.md)

Type vocabulary (closed set): Bug | Feature | Task

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy ISSUE-NNN flat namespace. New items **MUST** use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under HXC. Numeric portion is per-prefix (each prefix starts at 001). Legacy ISSUE-NNN IDs renamed forward-only per CONST-043; historical close-out narratives in docs/CONTINUATION.md preserve original IDs verbatim, and docs/Fixed.md annotates each closure with (ex-**ISSUE-NNN**) for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule,	this repo

Prefix	Scope	Source
	governance, infrastructure)	
HXA	HelixAgent submodule	submodules/ helix_agent (when present) / helix_agent/ tree
HXM	HelixMemory submodule	submodules/ helix_memory
HXL	HelixLLM submodule	submodules/helix_llm
HXQ	HelixQA submodule	submodules/helix_qa (helix_qa/)
HXS	HelixSpecifier submodule	submodules/ helix_specifier
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	submodules/ llm_orchestrator
HXV	HelixVerifier (= LLMsVerifier) submodule	submodules/ llms_verifier
HXD	HelixDocProcessor (= DocProcessor) submodule	submodules/ doc_processor
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic- digital)	submodules/planning
VEN	VisionEngine submodule (HelixDevelopment)	submodules/ vision_engine
SLF	SelfImprove submodule (vasic- digital)	submodules/ self_improve
STO	Storage submodule (vasic-digital)	submodules/storage
OPS	LLMOps submodule (vasic-digital)	submodules/llm_ops

Prefix	Scope	Source
VDB	VectorDB submodule (vasic-digital)	submodules/vector_db
OBS	Observability submodule (vasic-digital)	submodules/ observability
MCP	MCP_Module submodule (vasic-digital)	submodules/ mcp_module
MSG	Messaging submodule (vasic-digital)	submodules/messaging
MDW	Middleware submodule (vasic-digital)	submodules/ middleware
PLG	Plugins submodule (vasic-digital)	submodules/plugins
STR	Streaming submodule (vasic-digital)	submodules/streaming
WAT	Watcher submodule (vasic-digital)	submodules/watcher
CNV	conversation submodule (vasic-digital)	submodules/ conversation
AUT	Auth submodule (vasic-digital)	submodules/auth
LZY	Lazy submodule (vasic-digital)	submodules/lazy
ATP	AutoTemp submodule (vasic-digital)	submodules/auto_temp
PLI	PliniusCommon submodule (vasic-digital)	submodules/ plinius_common
CHL	challenges submodule (vasic-digital)	challenges/
CNT		containers/

Prefix	Scope	Source
	containers submodule	
SEC	security submodule	security/
PAN	panoptic submodule	panoptic/

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. `Watcher` → `WAT`). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine helix-gitlab URL
ISSUE-002	VEN-002	VisionEngine vasic-digital-github fork divergent
ISSUE-003	HXL-001	HelixLLM analysis_test.go hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON WriteTOON 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa TestPerformance flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

HXC-151 — Residual replica-RED branches in internal/server regression guards cannot fail, so they guard nothing

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Several regression tests in the server package carry a RED_MODE switch whose job is to prove the test can actually catch the bug it guards. In these cases the RED half checks a private copy of the old code that lives inside the test file itself, instead of the real shipped code. Because that copy is part of the test and never changes when the product changes, the check produces the same result on every build ever made — the working one, the broken one, and any future one — so it can never fail and proves nothing about the product. This matters because everyone reads a green result as evidence that the guard is real, when the guard is in fact inert: the original defect could return and the suite would stay green. The confirmed cases are `handlers_systemstatus_guard_test.go` (calls a test-local `preFixSystemStatusHealthCheck` helper instead of the real `getSystemStatus` handler, so the nil-database guard in the product is bypassed by construction), `handlers_qastatus_deadlock_guard_test.go` (hand-rolls the locking sequence in `reproduceRecursiveRLockDeadlock` rather than calling the shipped handler, so it only re-proves a standard Go library property), `llm_rag_test.go` (deliberately skips the real handler and its retrieval step, so the ‘prompt was not augmented’ result is guaranteed by the test’s own omission), and `llm_working_funnel_test.go` (one case has no RED branch at all, and another calls a real function that the fix never touched). To reproduce: revert the matching product fix and re-run the test with `RED_MODE=1`; the branch still passes, which it must not. The work is to rewrite each RED branch so it drives the real shipped code path, exactly as was done for the three sibling guards in `llm_default_model_regression_test.go` and `llm_generate_regression_test.go`. Done means that for every listed test, `RED_MODE=1` passes on an artifact with only its own fix reverted and fails on the current artifact, that this four-way result is captured as evidence, and that a repeat of the enumerated search finds no remaining test-local stand-ins.

HXC-152 — `llm_working_funnel_test.go` defaults to RED mode, so its fix-verifying assertions never run and it still reports success

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude

One test file in the server package chooses between ‘reproduce the old bug’ mode and ‘confirm the fix works’ mode by reading an environment variable, but its default is backwards: when the variable

is unset — which is how the suite runs normally and in every automated run — it selects the old-bug mode. The practical effect is that the checks confirming the fixed behaviour never execute at all, while the test still reports success, so the team sees a green result for behaviour nobody actually verified. The behaviour left unguarded is the model-listing funnel that is supposed to hide models which failed verification, scored below the quality threshold, or have no API key configured, which is precisely the anti-bluff filtering the original fix was written to provide. A second test in the same file simply skips itself outright under the same default, so the provider key-gate is unguarded too. Every other polarity switch in this package defaults the opposite way, so this file is also inconsistent with its neighbours and will mislead anyone who assumes the shared convention. To reproduce: run the package normally and observe that the guarded assertions are never reached while both tests report PASS. Done means the default is flipped so the fix-verifying assertions run in ordinary runs, the reproduce-the-old-bug mode still works when explicitly requested, and captured evidence shows those assertions genuinely execute and genuinely fail when the filtering is removed.